

Apache Paimon Compact

模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：Compact (paimon-core)

分支版本：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月29日

第1章 模块概述

1.1 模块定位

compact 包位于 paimon-core 模块下的 org.apache.paimon.compact 包中，是 Apache Paimon 数据湖存储引擎的核心 Compaction 抽象层。该模块定义了 Compaction（合并压缩）操作的顶层接口、抽象基类与数据模型，为 MergeTree 和 Append-Only 两种存储模式下的文件合并提供统一的编程范式。

Compaction 是 LSM-Tree 存储引擎的核心操作之一。随着数据持续写入，小文件会不断积累，导致读取放大（Read Amplification）和空间放大（Space Amplification）问题。Compact 模块通过异步合并、策略驱动的方式，将多个小文件合并为更大的有序文件，从而在写放大、读放大和空间放大之间取得平衡。

1.2 核心目标

- 统一抽象：为 MergeTree（主键表）和 AppendOnly（追加表）两种 FileStore 提供统一的 Compaction 管理接口
- 异步执行：通过 Future 机制支持 Compaction 任务在独立线程池中异步执行，不阻塞写入路径
- 策略可插拔：通过 CompactStrategy 接口支持 Universal Compaction、Clustering Compaction 等多种压缩策略
- 删除向量集成：通过 CompactDeletionFile 与 BucketedDvMaintainer 集成 Deletion Vectors，支持延迟/即时生成模式
- 指标可观测：与 CompactionMetrics 集成，支持 Compaction 耗时、文件大小、队列深度等核心指标上报

1.3 源码文件一览

文件名	行数	类型	职责描述
CompactManager.java	60	接口	Compaction 管理器顶层接口，定义触发、获取结果、取消等核心操作
CompactFutureManager.java	69	抽象类	基于 Future 的异步 Compaction 管理器基类，封装任务提交与结果获取
CompactTask.java	121	抽象类	Compaction 任务抽象，实现 Callable 接口，封装指标上报和模板方法
CompactResult.java	88	数据类	Compaction 结果封装，包含合并前/后文件列表、changelog 和删除文件
CompactUnit.java	64	数据类	Compaction 单元，封装待合并文件列表、输出 Level 和是否需要重写标记

文件名	行数	类型	职责描述
CompactDeletionFile.java	134	接口+实现	Compaction 删除文件管理，支持即时生成和延迟生成两种模式
NoopCompactManager.java	80	实现类	空操作 Compaction 管理器，用于 write-only 模式下跳过合并

第2章 核心架构设计

2.1 整体架构分层

Compact 模块采用经典的接口-抽象类-实现类三层架构，遵循面向接口编程原则。顶层 CompactManager 接口定义了统一的 Compaction 生命周期管理协议；中间层 CompactFutureManager 抽象类封装了异步任务管理的通用逻辑；底层由具体的 Manager 实现类针对不同存储模式提供差异化的合并策略。

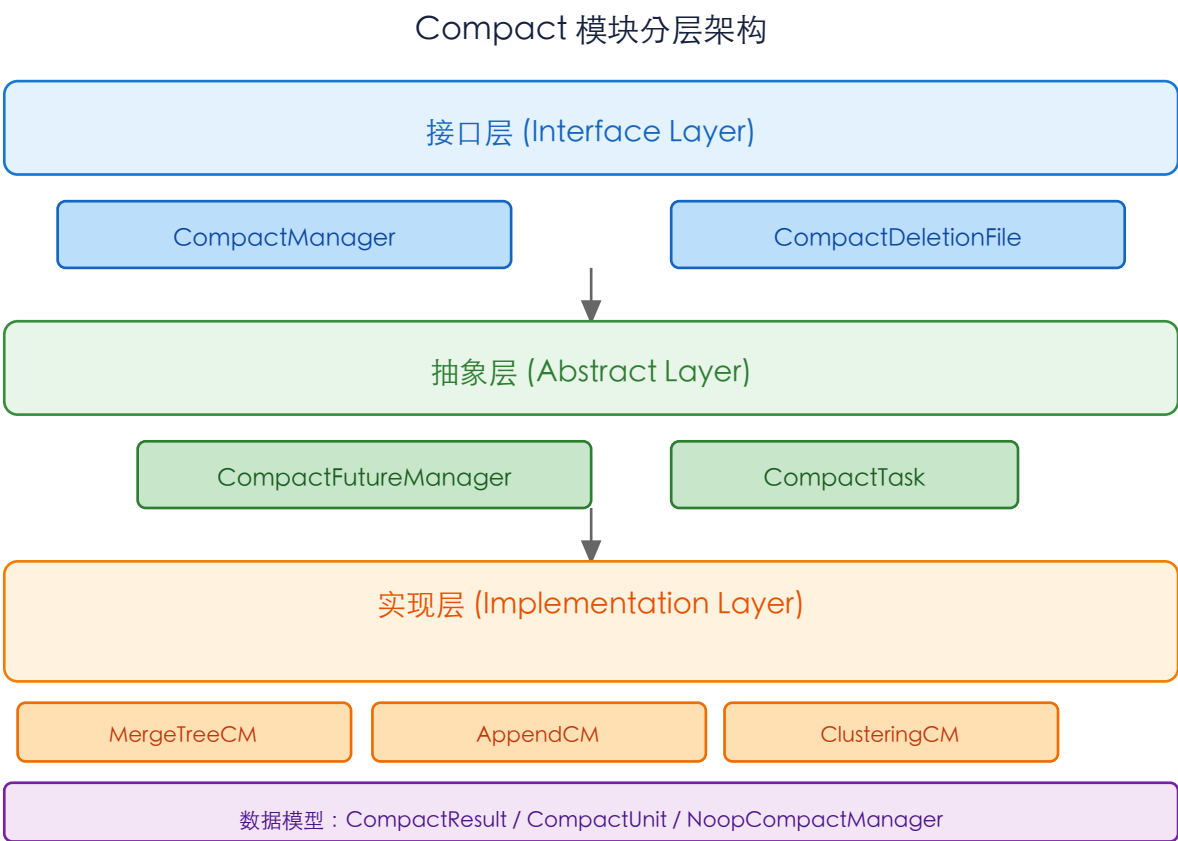


图2-1 Compact 模块分层架构图

2.2 类继承关系

CompactManager 接口继承了 java.io.Closeable，保证资源可以被正确释放。CompactFutureManager 是实现 CompactManager 接口的抽象基类，通过 protected 的 taskFuture 字段和模板方法 innerGetCompactionResult() 统一管理异步任务生命周期。三个具体实现分别覆盖不同的使用场景：

实现类	继承关系	适用场景
MergeTreeCompactManager	extends CompactFutureManager	主键表 (KeyValueFileStore)，基于 Levels 结构的 LSM-Tree Compaction

实现类	继承关系	适用场景
BucketedAppendCompactManager	extends CompactFutureManager	追加表（AppendOnlyFileStore），基于文件大小的合并策略
ClusteringCompactManager	extends CompactFutureManager	主键表的聚簇 Compaction，按聚簇列排序和分区合并
BucketedAppendClusterManager	extends CompactFutureManager	追加表的聚簇 Compaction，基于增量聚簇策略
NoopCompactManager	implements CompactManager	write-only 模式，所有操作为空实现

2.3 外部依赖关系

compact 包的核心外部依赖如下：

依赖组件	所属包	关系说明
DataFileMeta	o.a.p.io	数据文件元信息，Compaction 的输入/输出单元
BucketedDvMaintainer	o.a.p.deletionvectors	删除向量维护器，管理文件级删除标记
DeletionVectorsIndexFile	o.a.p.deletionvectors	删除向量索引文件读写
IndexFileMeta	o.a.p.index	索引文件元信息
CompactionMetrics.Reporter	o.a.p.operation.metrics	Compaction 指标上报器
LevelSortedRun	o.a.p.mergetree	按 Level 组织的有序文件集合
CompactStrategy	o.a.p.mergetree.compact	Compaction 策略接口
CoreOptions	o.a.p	配置项（write-only 等）

第3章 核心类设计模式详解

3.1 CompactManager — 策略模式 (Strategy Pattern)

CompactManager 接口本身就是策略模式的体现。它定义了 Compactation 操作的统一契约（triggerCompaction、getCompactionResult、cancelCompaction 等），而具体的合并策略由不同的实现类提供。调用方（如 MergeTreeWriter、AppendOnlyWriter）通过接口引用持有 CompactManager，无需关心底层是 MergeTree 合并还是 Append-Only 合并。

关键方法：

- triggerCompaction(boolean fullCompaction) — 触发一次 Compactation，fullCompaction 参数控制是否全量合并
- getCompactionResult(boolean blocking) — 获取 Compactation 结果，支持阻塞/非阻塞两种模式
- shouldWaitForLatestCompaction() — 判断写入是否需要等待当前 Compactation 完成（背压控制）
- addNewFile(DataFileMeta) — 向管理器注册新写入的数据文件

设计意图：通过接口多态实现开闭原则，新增 Compactation 策略只需增加新的实现类，无需修改调用方代码。

3.2 CompactFutureManager — 模板方法模式 (Template Method Pattern)

CompactFutureManager 是典型的模板方法模式实现。它定义了异步 Compactation 管理的骨架算法（innerGetCompactionResult 方法），将可变部分（如何触发任务、如何处理结果）延迟到子类实现。

模板方法：

- innerGetCompactionResult(boolean blocking) — 定义获取结果的标准流程：检查 Future 状态 → 阻塞/非阻塞等待 → 处理取消异常 → 返回结果
- cancelCompaction() — 提供统一的取消逻辑，通过 Future.cancel(true) 中断线程
- compactNotCompleted() — 通过 taskFuture != null 判断是否有未完成任务

钩子方法（子类覆盖）：

- obtainCompactResult() — 可被子类覆盖以自定义结果获取行为（如 MockFailResultCompactionManager 用于测试）

设计意图：将异步任务管理的不变逻辑（Future 状态检查、异常处理）抽取到基类，减少子类中的重复代码。

3.3 CompactTask — 模板方法模式 + 观察者模式

CompactTask 实现了 Callable<CompactResult> 接口，其 call() 方法是一个完整的模板方法，定义了 Compaction 任务执行的标准流程：启动计时 → 执行合并 (doCompact) → 上报指标 → 停止计时 → 减少队列计数。子类只需实现 doCompact() 抽象方法即可。

模板方法流程：

- 1. startTimer() — 启动 Compaction 计时器
- 2. doCompact() — 【抽象方法】执行实际的文件合并逻辑
- 3. 上报指标：compactionTime、completedCount、inputSize、outputSize
- 4. stopTimer() + decreaseCompactionsQueuedCount() — 清理计时器和队列计数

已知子类：MergeTreeCompactTask、FileRewriteCompactTask、FullCompactTask、AutoCompactTask、BucketedAppendClusterTask

3.4 CompactDeletionFile — 策略模式 + 延迟初始化模式 (Lazy Initialization)

CompactDeletionFile 接口定义了两种删除文件生成策略，通过静态工厂方法创建：

- GeneratedDeletionFile（即时生成）：异步 Compaction 完成后立即生成删除向量索引文件。适用于异步 Compaction 场景，因为任务完成时需要立即产出文件元信息。
- LazyCompactDeletionFile（延迟生成）：仅在 prepareCommit 调用 getOrCompute() 时才真正生成文件。适用于同步 Compaction 场景，避免不必要的文件写入。

mergeOldFile 方法：用于处理异步 Compaction 结果更新时的旧删除文件合并。当新的删除文件存在时，清理旧文件（释放存储空间）；当新文件为 null 时，保留旧文件。该方法包含状态校验，防止已被消费 (getInvoked/generated) 的文件被错误合并。

3.5 NoopCompactManager — 空对象模式 (Null Object Pattern)

NoopCompactManager 是经典的空对象模式实现。当表配置为 write-only 模式时，CompactManager 被替换为此空实现，所有方法返回安全的默认值 (false、空列表、Optional.empty())，无需调用方额外判空。

特殊行为：triggerCompaction(true) 会抛出异常并提示用户设置 write-only=false，这是一种防御性编程手段，防止在 write-only 模式下意外触发全量 Compaction。

3.6 CompactUnit — 值对象模式 (Value Object) + 静态工厂方法

CompactUnit 是不可变的值对象，封装了一次 Compaction 操作的输入参数：待合并文件列表（files）、输出 Level（outputLevel）和是否需要文件重写（fileRewrite）。

静态工厂方法：

- fromLevelRuns(int outputLevel, List<LevelSortedRun> runs) — 从 LevelSortedRun 构建，将所有 Run 的文件展平
- fromFiles(int outputLevel, List<DataFileMeta> files, boolean fileRewrite) — 直接从文件列表构建

设计意图：CompactUnit 由 CompactStrategy.pick() 产出，作为 Compaction 决策的载体传递给 CompactTask 执行。

第4章 核心流程图

4.1 Compaction 触发与执行流程

以下流程图展示了从 Writer 触发 Compaction 到获取结果的完整流程。这是整个 Compact 模块最核心的工作流，适用于 MergeTree 和 Append 两种模式。

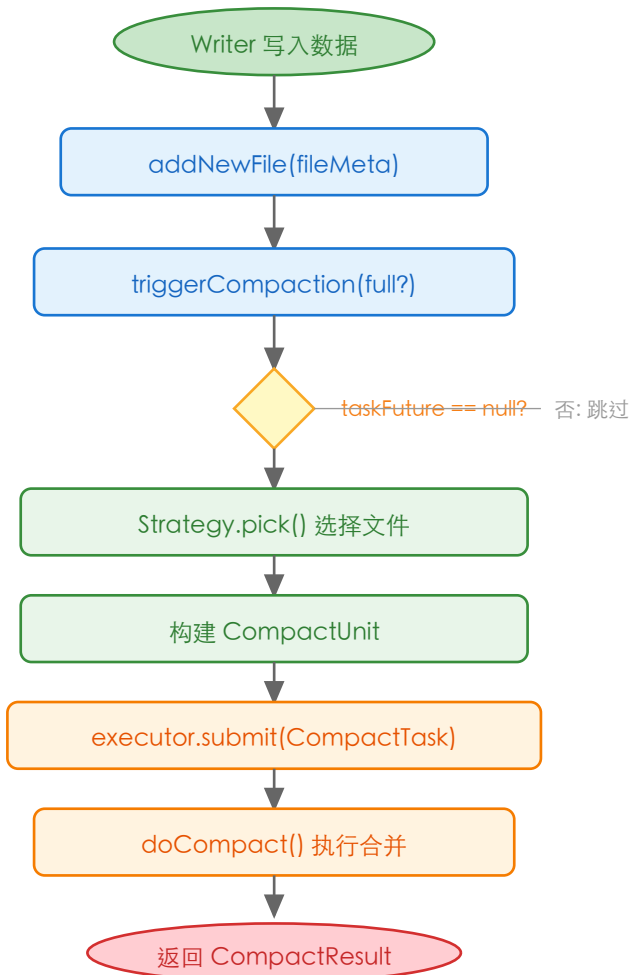


图4-1 Compaction 触发与执行流程图

4.2 CompactResult 获取与更新流程

当 Writer 需要 prepareCommit 或检测到 Compaction 完成时，通过 getCompactionResult 获取结果并更新文件层级。

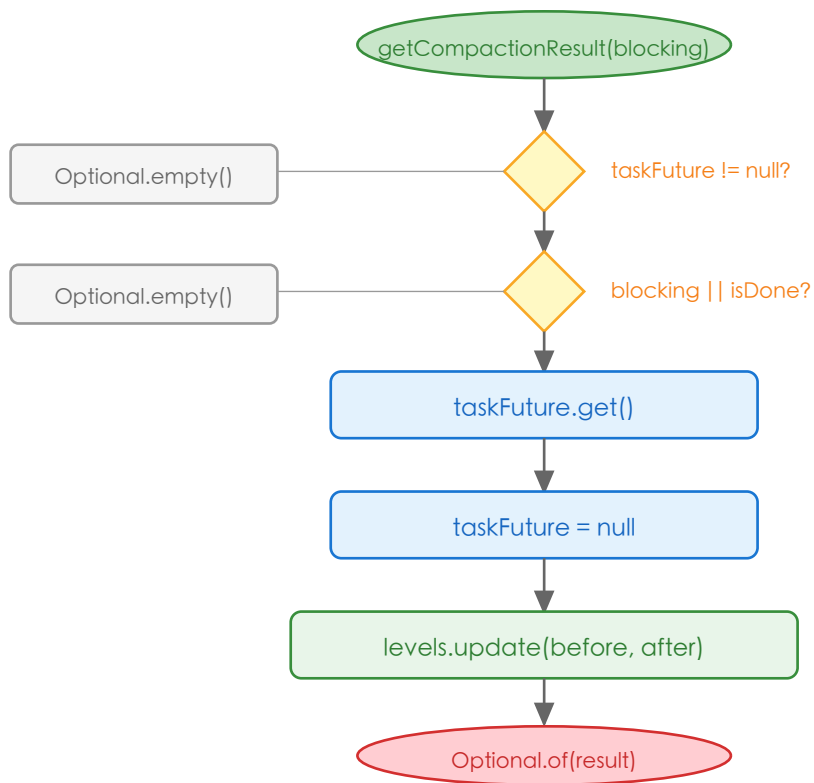


图4-2 CompactResult 获取与 Level 更新流程

4.3 CompactDeletionFile 生成策略流程

删除文件的生成分为即时（GeneratedDeletionFile）和延迟（LazyCompactDeletionFile）两种模式，取决于 Compaction 是同步还是异步执行。

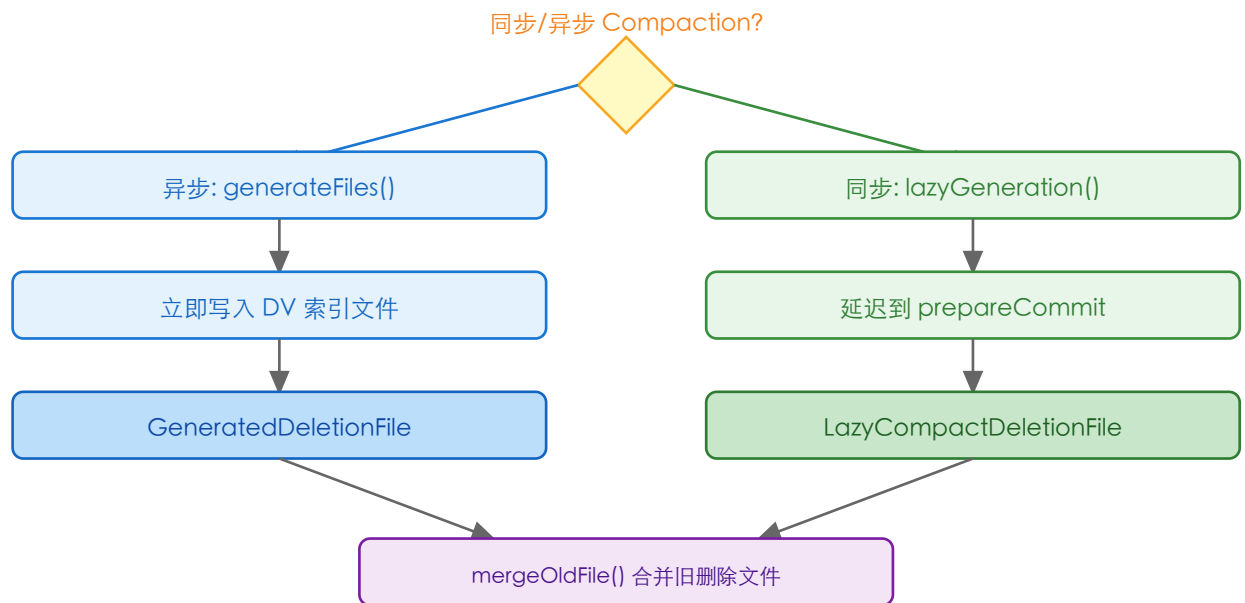


图4-3 CompactDeletionFile 生成策略流程图

第5章 核心时序图

5.1 MergeTree Compaction 执行时序

以下时序图展示 MergeTreeWriter 触发 Compaction 到结果回收的完整交互过程。涉及 MergeTreeWriter、MergeTreeCompactManager、CompactStrategy、ExecutorService 和 MergeTreeCompactTask 五个参与者。

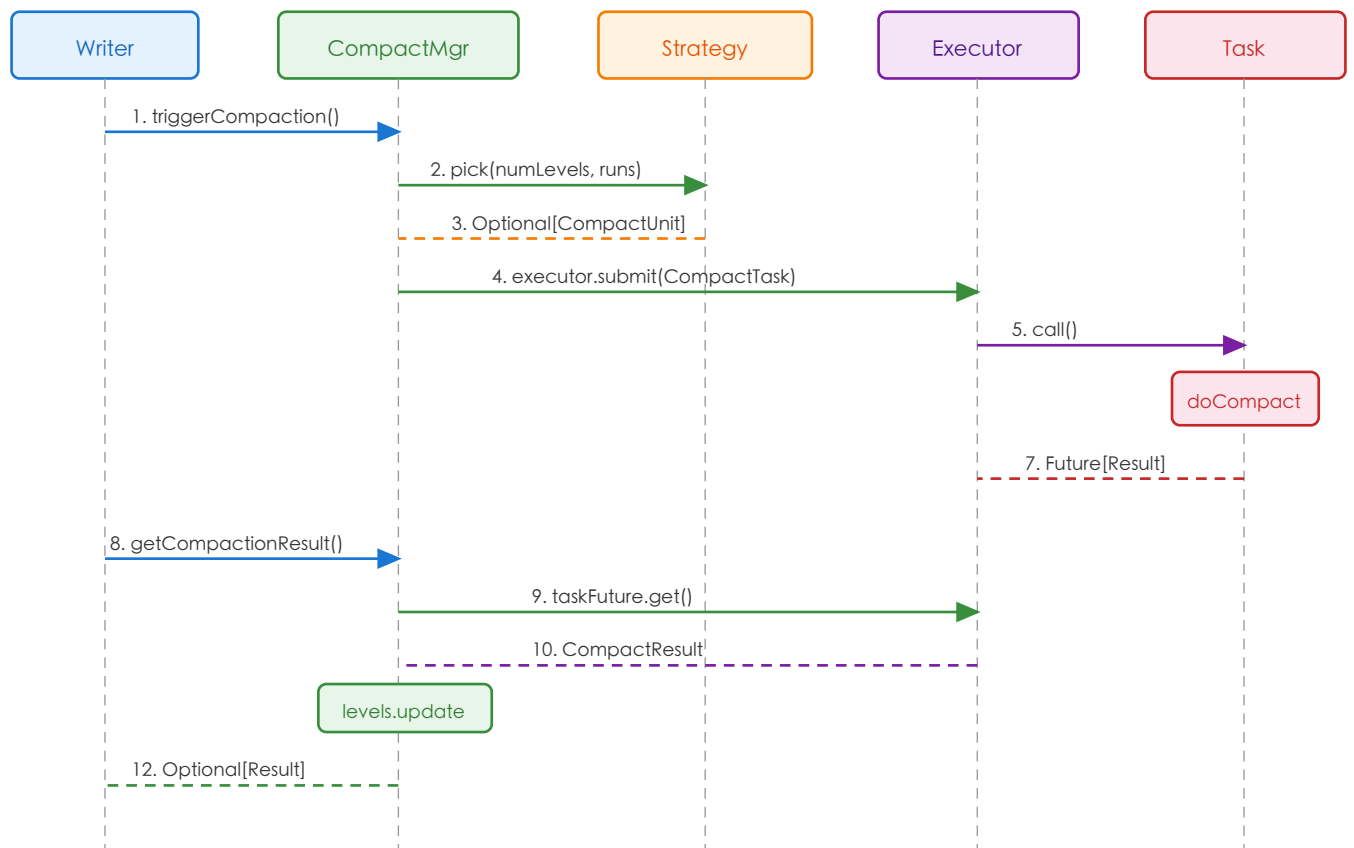


图5-1 MergeTree Compaction 执行时序图

5.2 CompactDeletionFile 生命周期时序

以下时序图展示 CompactDeletionFile 在异步 Compaction 场景下从创建、合并到消费的完整生命周期。

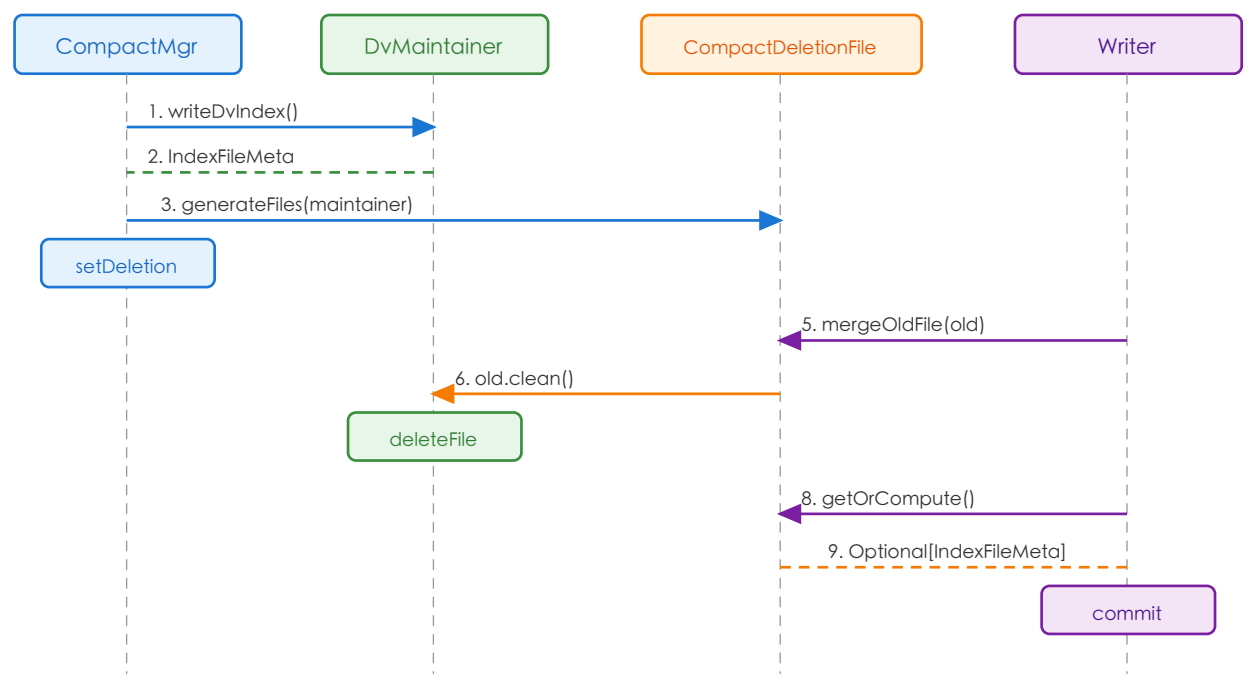


图5-2 CompactDeletionFile 生命周期时序图（异步模式）

第6章 关键场景调用链

6.1 场景一：MergeTree 普通 Compaction 触发

触发条件：MergeTreeWriter 写入新数据文件后，当 SortedRun 数量超过 numSortedRunCompactionTrigger 时自动触发。

调用链路：

步骤	类.方法	说明
1	MergeTreeWriter.flushWriteBuffer()	将内存缓冲区刷写到数据文件
2	MergeTreeCompactManager.addNewFile()	注册新文件到 Levels 结构
3	MergeTreeCompactManager.triggerCompaction(false)	触发普通 Compaction
4	CompactStrategy.pick(numLevels, runs)	UniversalCompaction 选择待合并的 SortedRun
5	CompactUnit.fromLevelRuns(outputLevel, runs)	构建 CompactUnit 对象
6	MergeTreeCompactManager.submitCompaction(unit, dropDelete)	提交 Compaction 任务
7	MergeTreeCompactTask.call()	异步线程执行，调用 doCompact()
8	IntervalPartition.partition()	将文件按 key 区间分区
9	CompactRewriter.rewrite(outputLevel, dropDelete, sections)	执行实际文件合并
10	CompactResult 返回	包含 before/after 文件列表

6.2 场景二：MergeTree 全量 Compaction

触发条件：用户显式调用 fullCompaction，或 EarlyFullCompaction 策略判断需要全量合并。

调用链路：

步骤	类.方法	说明
1	MergeTreeCompactManager.triggerCompaction(true)	触发全量 Compaction
2	CompactStrategy.pickFullCompaction()	选择所有文件，包含 recordLevelExpire/DV 检查

步骤	类.方法	说明
3	CompactUnit.fromLevelRuns(maxLevel, runs)	输出到最高 Level
4	MergeTreeCompactManager.submitCompaction(unit, true)	dropDelete=true，清理删除标记
5	MergeTreeCompactTask.doCompact()	执行全量合并
6	CompactDeletionFile.generateFiles/lazyGeneration()	根据同步/异步模式生成删除文件

6.3 场景三：Append-Only 自动 Compaction

触发条件：追加表写入后，当小文件数量超过 minFileNum 时自动触发合并。

调用链路：

步骤	类.方法	说明
1	BucketedAppendCompactManager.triggerCompaction(false)	触发自动 Compaction
2	pickCompactBefore()	从 PriorityQueue 中按 minSequenceNumber 选择候选文件
3	AutoCompactTask 构建	将候选文件和 CompactRewriter 封装为任务
4	executor.submit(task)	提交到线程池异步执行
5	AutoCompactTask.doCompact()	调用 rewriter.rewrite() 合并文件
6	dvMaintainer.removeDeletionVectorOf()	清理被合并文件的删除向量
7	CompactDeletionFile.generateFiles()	生成新的删除向量索引文件

6.4 场景四：Compaction 结果回收与 Commit

触发条件：Writer.prepareCommit() 被调用时，需要获取 Compaction 结果并构建提交增量。

调用链路：

步骤	类.方法	说明
1	MergeTreeWriter.prepareCommit(blocking)	触发提交准备
2	CompactManager.getCompactionResult(blocking)	获取 Compaction 结果
3	CompactFutureManager.innerGetCompactionResult()	检查 Future 状态，等待/获取结果
4	CompactResult.before()/after()	获取合并前后的文件列表
5	CompactDeletionFile.getOrCompute()	获取/延迟生成删除文件索引
6	updateCompactResult()	合并旧的删除文件（mergeOldFile）
7	构建 CompactIncrement	封装 compactBefore、compactAfter、changelog 和 indexFiles

步骤	类.方法	说明
8	CommitMessageImpl 返回	作为 Snapshot 提交的一部分

6.5 场景五：Write-Only 模式下的 Compaction 拦截

触发条件：表配置 write-only=true 时，CompactManager 被替换为 NoopCompactManager。

调用链路：

步骤	类.方法	说明
1	MergeTreeCompactManagerFactory.create()	检查 options.writeOnly()
2	return new NoopCompactManager()	使用空对象替代真实管理器
3	Writer.triggerCompaction(false)	调用 NoopCompactManager.triggerCompaction(false)，空操作
4	Writer.triggerCompaction(true)	抛出异常，提示不支持 full compaction
5	Writer.getCompactionResult()	返回 Optional.empty()，无合并结果

6.6 场景六：Clustering Compaction 两阶段合并

触发条件：配置了 pkClusteringOverride 的主键表，有未排序（Level 0）文件时触发。

调用链路：

步骤	类.方法	说明
1	ClusteringCompactManager.triggerCompaction()	检查是否有 Level 0 文件
2	executor.submit(CompactTask)	提交包含 compact() 的匿名任务
3	Phase 1: sortAndRewriteFiles()	将 Level 0 文件按聚簇列排序并重写
4	keyIndex.updateIndex()	更新聚簇键索引
5	Phase 2: pickMergeCandidates()	基于键区间重叠选择合并组
6	mergeAndRewriteFiles()	合并重叠的排序文件
7	dvMaintainer.removeDeletionVectorOf()	清理合并文件的删除向量
8	CompactDeletionFile 设置	根据 lazyGenDeletionFile 配置选择生成策略

第7章 配置与扩展

7.1 关键配置属性

compact 模块的行为受以下核心配置项影响（通过 CoreOptions 访问）：

配置项	影响	说明
write-only	选择 NoopCompact Manager	设为 true 时跳过所有 Compaction
num-sorted-run.stop-trigger	背压控制	SortedRun 数量超过此值时写入等待
num-sorted-run.compaction-trigger	触发合并	SortedRun 数量超过此值时触发合并
compaction.max-size-amplification-percent	全量合并判断	空间放大百分比阈值
compaction.size-ratio	增量合并判断	大小比率阈值
target-file-size	输出文件大小	合并后的目标文件大小
deletion-vectors.enabled	DV 集成	启用删除向量，影响 CompactDeletionFile 生成
prepare-commit-wait-compaction	延迟生成 DV	控制 CompactDeletionFile 使用 lazy/generated 模式

7.2 扩展点

compact 模块提供以下扩展点，可用于自定义 Compaction 行为：

- CompactManager 接口：实现自定义 Compaction 管理器，覆盖文件管理和触发逻辑
- CompactTask 抽象类：通过实现 doCompact() 方法自定义合并逻辑
- CompactStrategy 接口：实现自定义文件选择策略（如基于时间、基于大小等）
- CompactRewriter 接口：自定义文件重写逻辑，如添加过滤、转换或特殊编码
- KvCompactionManagerFactory：通过工厂模式选择不同的 CompactManager 实现

第8章 设计亮点与总结

8.1 设计亮点

- 清晰的接口抽象：CompactManager 接口将 Compaction 的生命周期（触发、等待、结果获取、取消）抽象为 6 个方法，足够简洁又覆盖了所有使用场景。两种 FileStore（KV/Append）共享同一套接口。
- 异步执行与背压控制：通过 CompactFutureManager 的 Future 机制实现非阻塞 Compaction，同时 shouldWaitForLatestCompaction/shouldWaitForPreparingCheckpoint 提供了优雅的回压控制，在 SortedRun 过多时让写入线程主动等待，防止文件无限膨胀。
- 双模式删除文件管理：CompactDeletionFile 的 Generated/Lazy 双模式设计精巧地解决了同步/异步 Compaction 场景下删除向量文件生成时机的差异。mergeOldFile 方法确保了异步场景下多次 Compaction 结果之间删除文件的正确合并和旧文件的及时清理。
- 指标全覆盖：CompactTask 的模板方法在不增加子类负担的情况下，统一上报了 compactionTime、completedCount、queuedCount、inputSize、outputSize 五大核心指标。
- 空对象模式的防御性设计：NoopCompactManager 不仅避免了 null 判断的散落，还在 triggerCompaction(true) 时主动抛出异常并给出清晰的修复建议，体现了良好的用户体验意识。

8.2 设计限制

- 取消操作可能泄漏文件：cancelCompaction() 方法的 TODO 注释指出，如果 Compaction 实际已完成但尚有 CPU 后处理工作，取消操作可能导致孤儿文件。
- CompactResult.merge 不支持 DeletionFile：当调用 merge 时如果已设置了 deletionFile，会直接抛出 UnsupportedOperationException，这限制了某些合并场景。
- 单一 Future 限制并发：CompactFutureManager 使用单个 taskFuture 字段，意味着同一时刻只能有一个 Compaction 任务在执行，无法支持多任务并行合并。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	接口-抽象类-实现类三层架构清晰，职责分明
设计模式运用	★★★★★	策略模式、模板方法、空对象、延迟初始化等多种模式协同使用
可扩展性	★★★★☆	接口设计良好，但单 Future 限制了并发扩展
错误处理	★★★★☆	CancellationException 处理完善，但取消泄漏问题待解决
指标可观测性	★★★★★	通过 CompactTask 模板方法统一上报核心指标

评估维度	评分	说明
代码可读性	★★★★★	命名规范，注释完整，方法粒度适当

总结：compact 包作为 Apache Paimon 的 Compaction 抽象层，以极其精简的 7 个文件（约 600 行代码）为上层的 MergeTree、Append-Only、Clustering 三大 Compaction 引擎提供了统一的接口和基础设施。其设计充分体现了 SOLID 原则，尤其是接口隔离原则和依赖倒置原则。模块的核心价值在于将 Compaction 操作的生命周期管理（触发、执行、结果回收、取消）与具体的合并策略解耦，使得新增合并策略（如 Clustering Compaction）时只需新增实现类，无需修改任何已有代码。